Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan
Département d'Informatique
Filière : Ingénierie Informatique et Développement Logiciel (2IDL)
Année universitaire : 2026- 20267

Module M353 : Analyse et Conception Orientées Objet en Langage UML

Chapitre 3 : Diagramme de classes & Diagramme d'objets

Pr. Abderrahim EL MHOUTI

a.elmhouti@uae.ac.ma

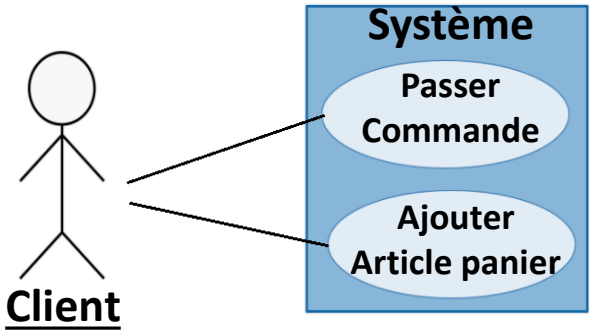
Plan du chapitre 3 : Diagramme de classes et d'objets

- Diagramme de classes
  - Introduction
  - Les classes
  - Relations entre classes
  - Méthodes abstraites
  - Classes abstraites
  - Interfaces
- Diagramme d'objets
- Élaboration d'un diagramme de classes

Diagramme de classes
(Class Diagram)

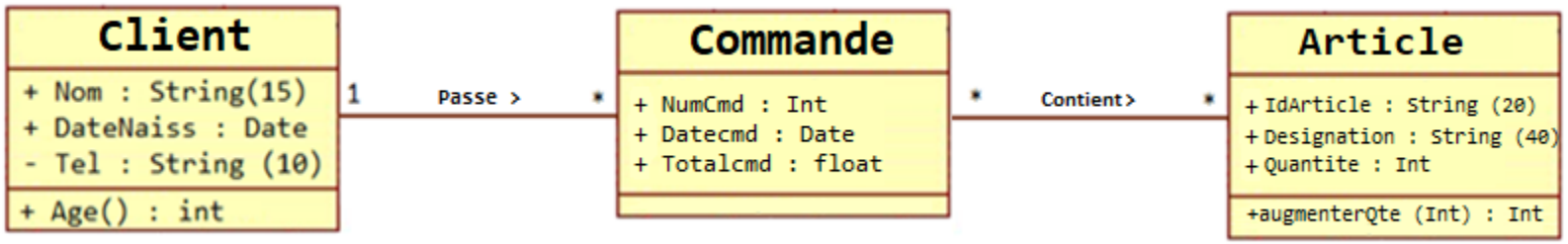
Diagramme de classes
Introduction

- Le **diagramme de CU** modélise à **QUOI** sert le système.
- Un système est composé d'**objets** qui interagissent entre eux et avec les **acteurs** pour réaliser ces CU.



- Le **diagramme de classes** décrit la **structure statique** du système en termes de **classes** qui le composent et de **relations** entre ces classes.

➔ il spécifie **QUI** sera à l'œuvre dans le système pour réaliser les **CU**.



Exemple simplifié de diagramme de classes

Diagramme de classes

Introduction

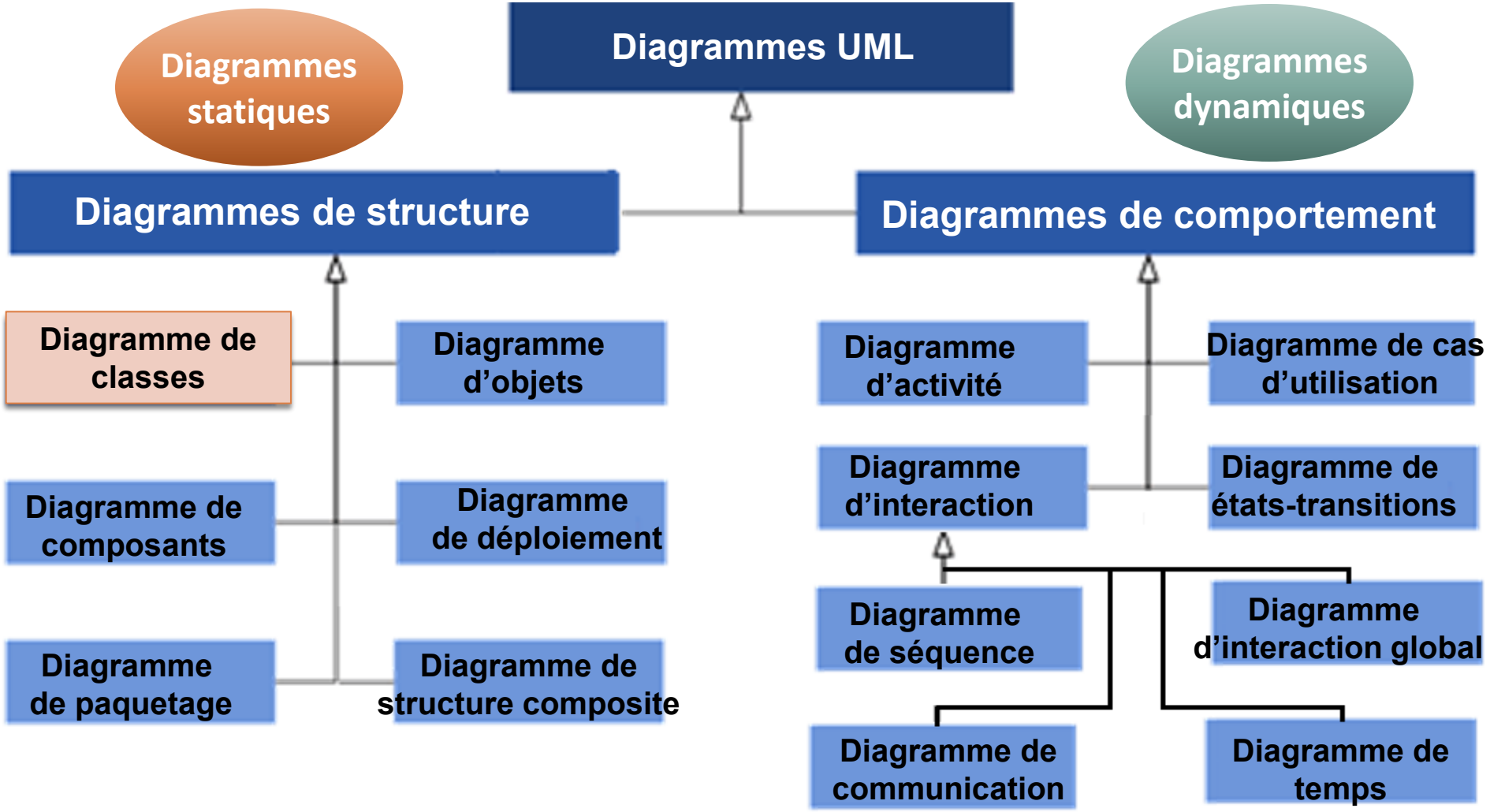
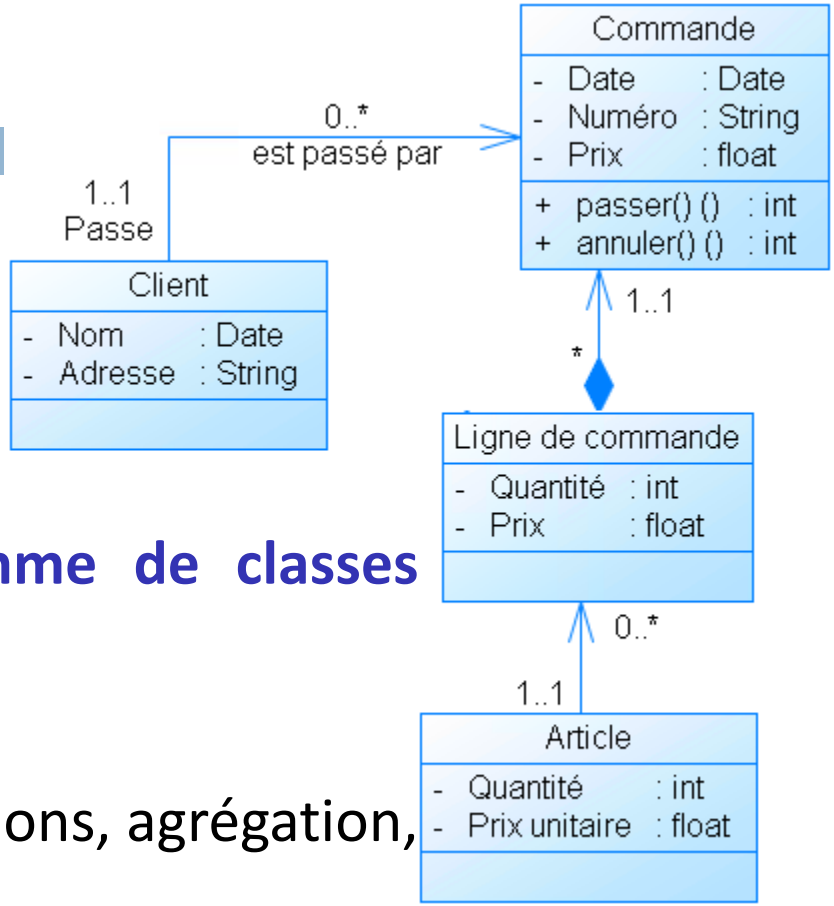


Diagramme de classes

Introduction

- Le **diagramme de classes** fournit une **représentation abstraite** des **objets** du système qui vont interagir pour réaliser les **CU**.



- Les principaux éléments du **diagramme de classes** sont:
  - les **classes** d'objets
  - les **relations** entre classes : associations, agrégation, composition, …
- Une **classe** est une description d'un ensemble d'objets ayant une structure commune.
- Un objet est une **instance** d'une classe.

Diagramme de classes

Remarques

- Le **diagramme de classes** permet de modéliser les **classes** du système et leurs **relations** **indépendamment** d'un **langage de programmation** particulier.

- Le **diagramme de classes** est considéré comme le **plus important** de la **modélisation OO** (il est le seul **obligatoire**).

- Deux analyses effectuées par deux personnes avec les mêmes spécifications peuvent aboutir à des **diagrammes de classes** différent mais peuvent être tous les deux bonnes.

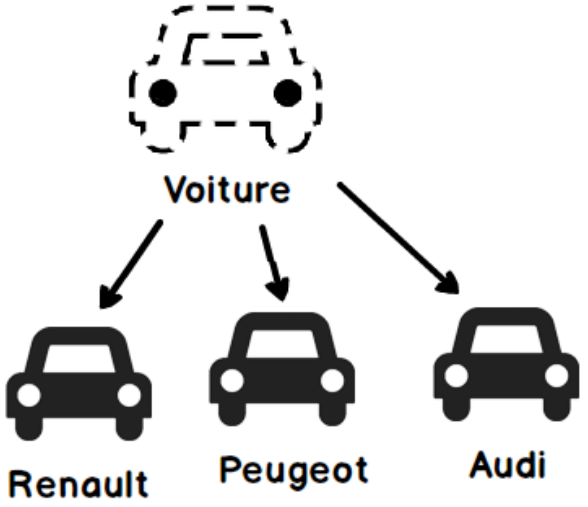
Classe

Classe

Notions de classe et d'instance de classe

- Une **classe** est un **concept** abstrait représentant des éléments du monde réel (exp: Voiture, Personne, Cours, …).

- Une **instance de classe** (objet) est la concrétisation d'un **concept abstrait** (d'une classe) **.**
  - ***Exemple*** **:**
    - **Concept** : Voiture
    - **Instances** : Renault, Peugeot, …



- Une **classe** spécifie la manière dont tous les **objets** de même type seront décrits (désignation, label, auteur, etc).

Classe

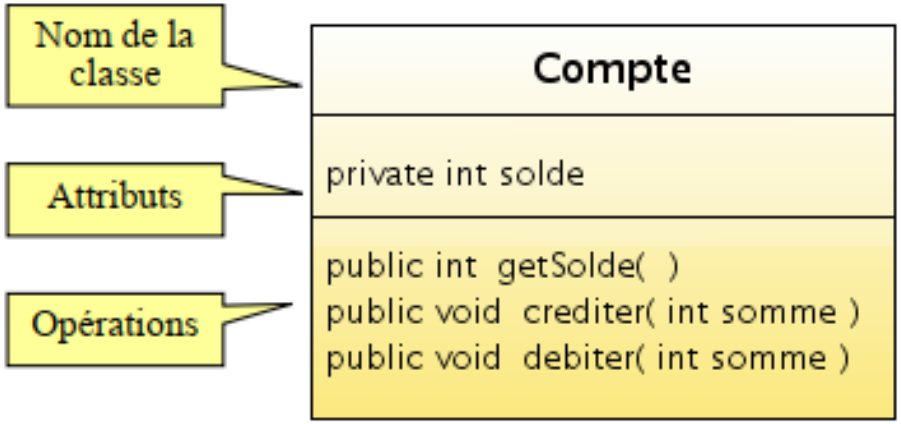
Notions de classe et d'instance de classe

- **Dans le système modélisé, un objet doit :**
  - être autonome
  - avoir une signification dans le contexte du système à modéliser
  - En relation avec d'autres objets

Classe

Caractéristiques d'une classe

- Une **classe** est la description d'un **ensemble** d'**objets** ayant des **caractéristiques communes :** des **attributs** et des **méthodes.**

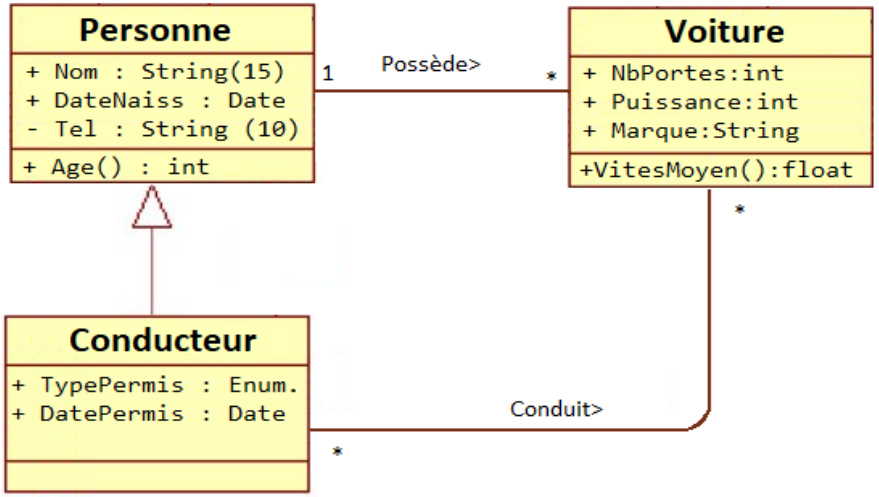


- Un **attribut** décrit une **donnée** de la classe. Il est caractérisé par son **type**, son **modificateur** d'accès, sa **valeur**, …
- Une **opération** est un service offert par la classe (un traitement que les objets correspondant peuvent effectuer).

Classe

Caractéristiques d'une classe

- Dans le **diagramme de classes**, les **terminaisons** d'**associations** sont considérées aussi comme **propriétés** qui peuvent faire partie des **caractéristiques** d'un objet (même titre que les **attributs** et les **méthodes**).



- **Classe** = **ensemble** d'**objets** ayant des **caractéristiques communes : attributs, méthodes, relations**.

Classe

Caractéristiques d'une classe

- La **classe** décrit alors :
  - L'**état de l'objet** : les **attributs** et généralement les ***terminaisons*** d'***associations***, tous deux réunis sous le terme de ***propriétés***.
    - ***attributs*** = valeurs de données pures : nombres, chaînes …
    - ***associations*** = liens entre classes du diagramme de classe.
  - Le **comportement de l'objet** : ***fonctions*** qui décrivent le comportement de l'objet ou produisent des résultats.

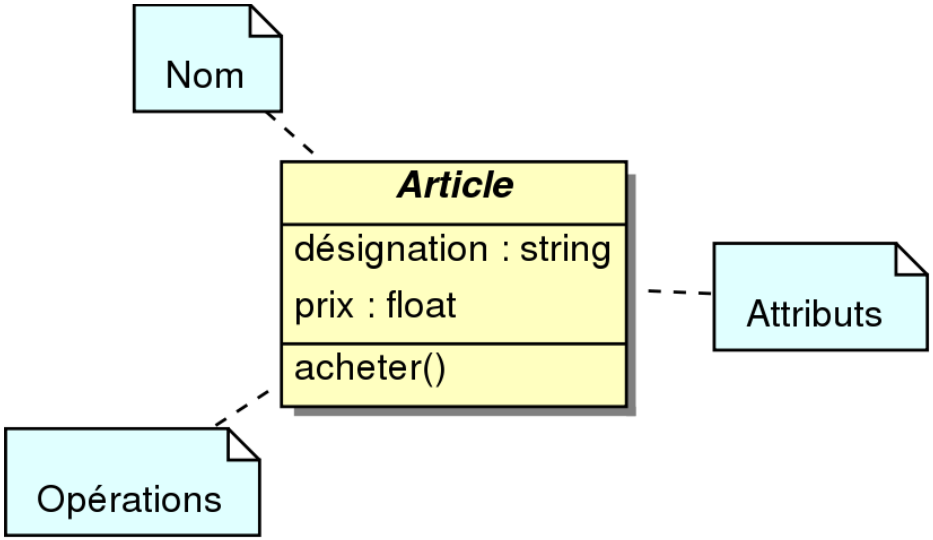


- **attributs** + **terminaisons d'association** + **fonctions** constituent les **caractéristiques** de la **classe** (et de ses instances).

Classe

Représentation graphique

- Une classe est un classeur. Elle est représentée par un **rectangle** divisé en trois à cinq **compartiments**.



- Le **nom** de la **classe** doit évoquer le concept décrit par la classe. Il commence par une majuscule.
- D'autres **compartiments** peuvent être ajoutés : **responsabilités**, **exceptions**, etc.

Classe

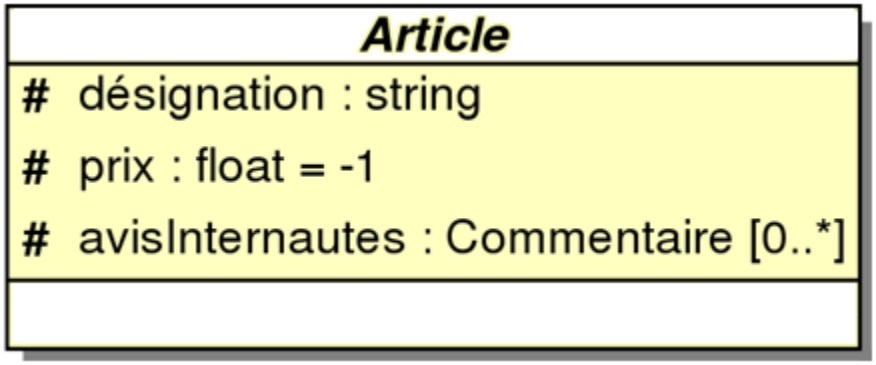
Compartiment des attributs

- Un **attribut** est définie par son **nom** et peut être **initialisé**. Sa **visibilité** est définie lors de sa déclaration.

- ***Syntaxe*** de la déclaration d'un attribut :

  modifAcces **nomAtt** : nomClasse [multi]= valeurInit

- ***Exemple*** : Attributs de la classe Article :



Classe

Compartiment des opérations

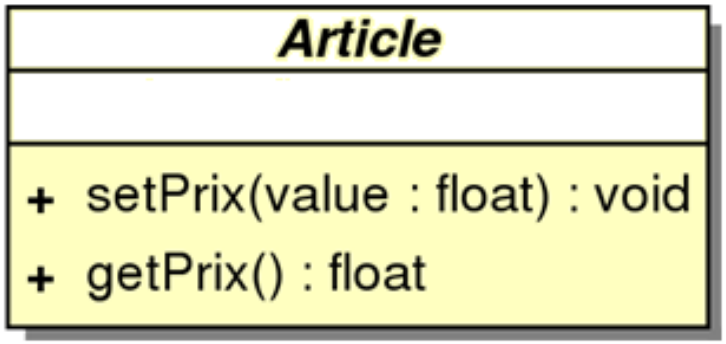
- Une **opération** est définie par son **nom** ainsi que par les **types** de ses **paramètres** et le **type de sa valeur de retour**.
- La syntaxe de la déclaration d'une opération :

  modifAcces **nomOperation** (parametres): TypeRetour

- La syntaxe de la liste des paramètres :

  nomParam1 : Type1, …, **nomParamN** : TypeN

- ***Exemple*** : Opérations de la classe Article :

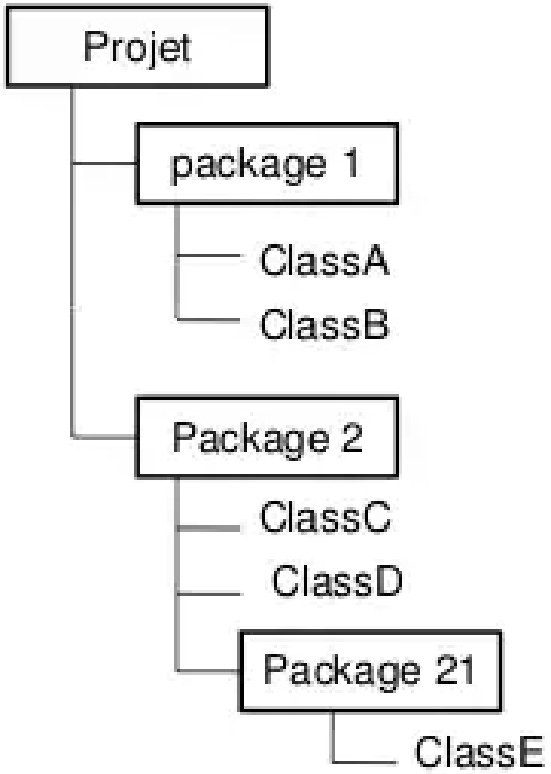


Classe

Encapsulation, visibilité, interface

- L'**encapsulation** permet d'empêcher l'accès aux **attributs** et **méthodes** de l'objet en définissant des **niveaux de visibilité.**

- La **visibilité** est la possibilité pour un élément de modélisation (hébergé dans une classe, un package, …) d'accéder à un autre élément hébergé ailleurs.

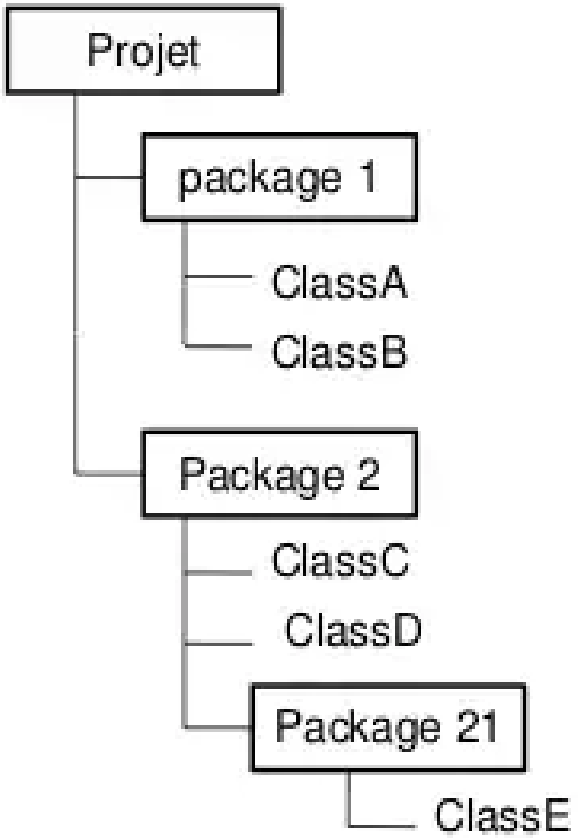


- L'**interface** de l'objet est la partie publique de l'objet (**données accessibles** aux utilisateurs de l'**objet**.

Classe

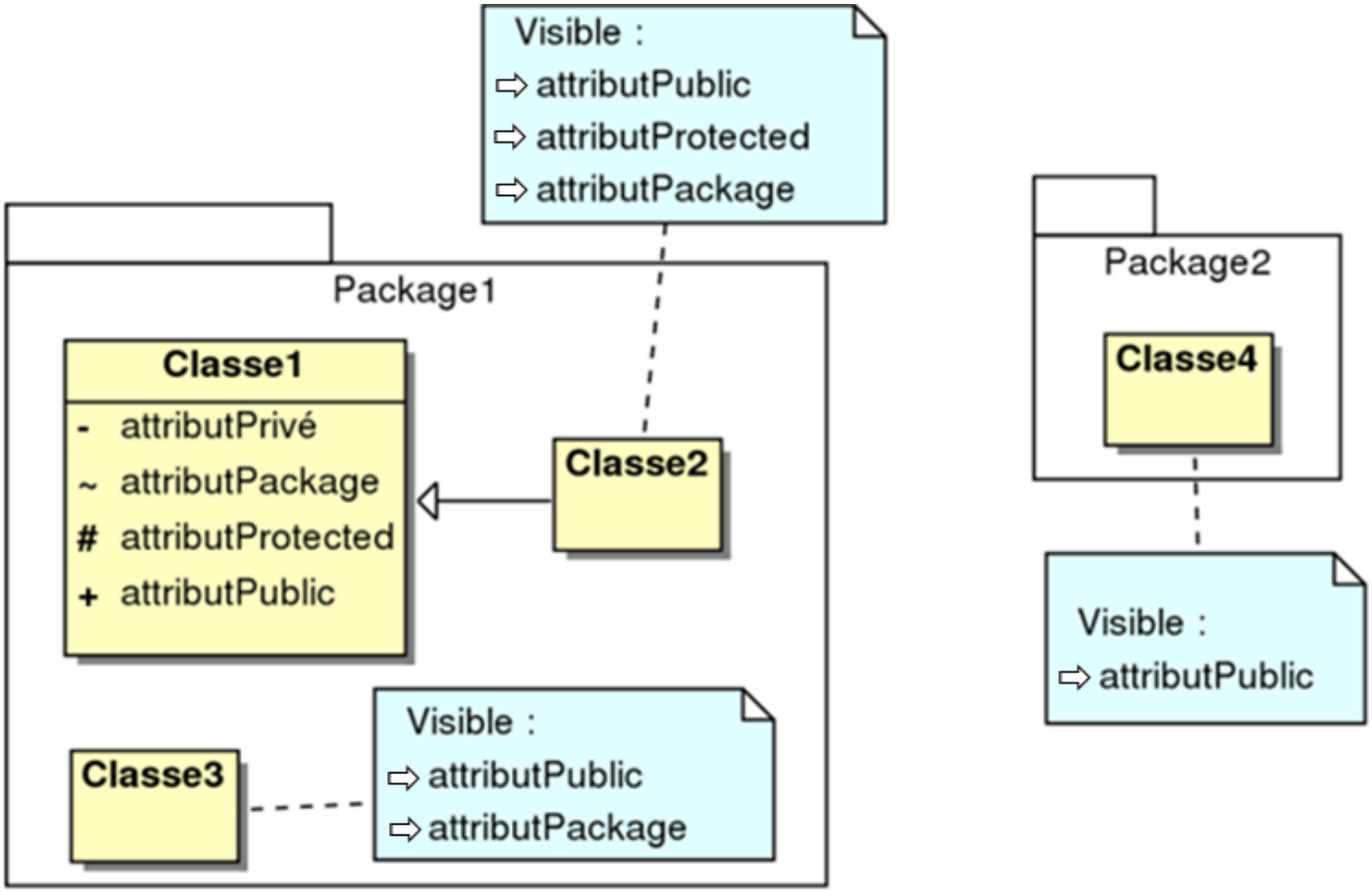
Encapsulation, visibilité, interface

- Il existe 4 niveaux de visibilités prédéfinies :
  - ***Public*** (**+**) : tout élément peut voir (accéder à) l'élément indiqué.
  - ***Protected*** (**#**) : seul un élément situé dans le conteneur ou un de ses descendants peut voir l'élément indiqué.
  - ***Private*** (**-**) : seul un élément situé dans le conteneur peut voir l'élément.
  - ***Package*** (**~** ou **rien** ou **default**) : seul un élément déclaré dans le même package peut voir l'élément.



Classe

Encapsulation : Exemple

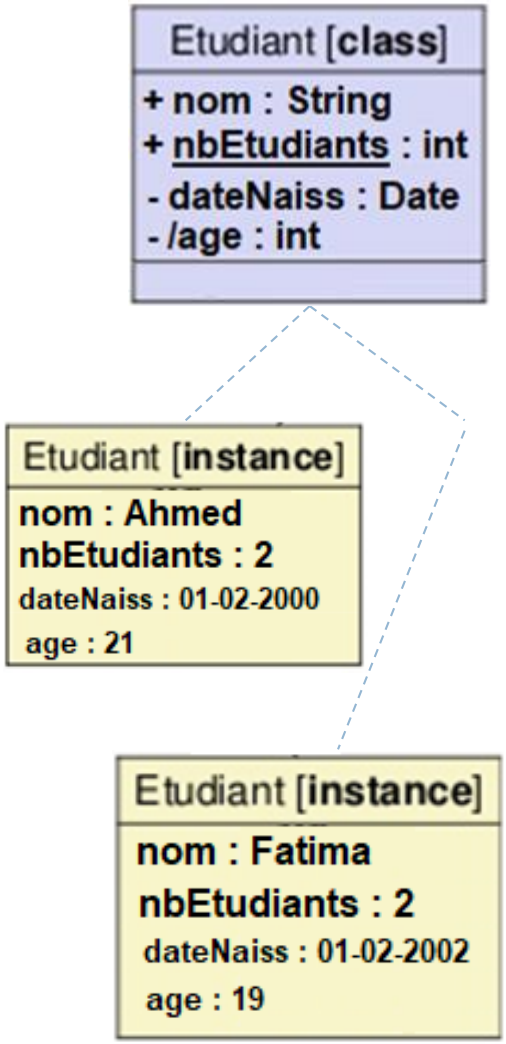


- NB : Les modificateurs d'accès sont également applicables aux opérations.

Classe

Types d'attributs

- **Attribut** (attribut d'instance) **:**
  - Attribut dont chaque objet possède sa propre valeur (les valeurs différent d'un objet à un autre).
- **Attribut de classe** (attribut statique) :
  - attribut qui garde une valeur unique et partagée par tous les objets de la classe. Graphiquement, un attribut de classe est souligné.
- **Attribut dérivé :**
  - attribut qui peut être calculé à partir d'autres attributs et de formules de calcul. Il est symbolisé par l'ajout d'un « / » devant leur nom.

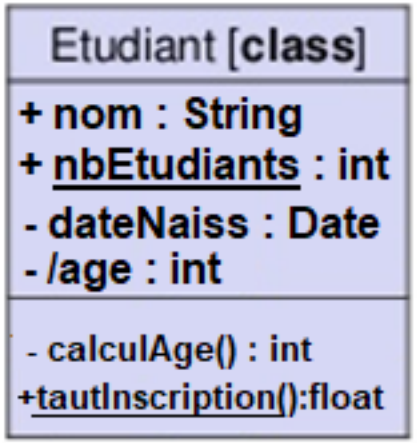


Classe

Types de méthodes

- **Méthode** (méthode d'instance):
  - Une méthode d'instance a pour but de manipuler les attributs de l'objet.

- **Méthode de classe** (méthode statique):
  - Une méthode de classe est une méthode qui n'agit pas sur des variables d'instance mais uniquement sur des **variables de classe**.
  - Elle peut être appelée sans avoir instancié la classe
  - Graphiquement, elle est soulignée.



Relations entre classes

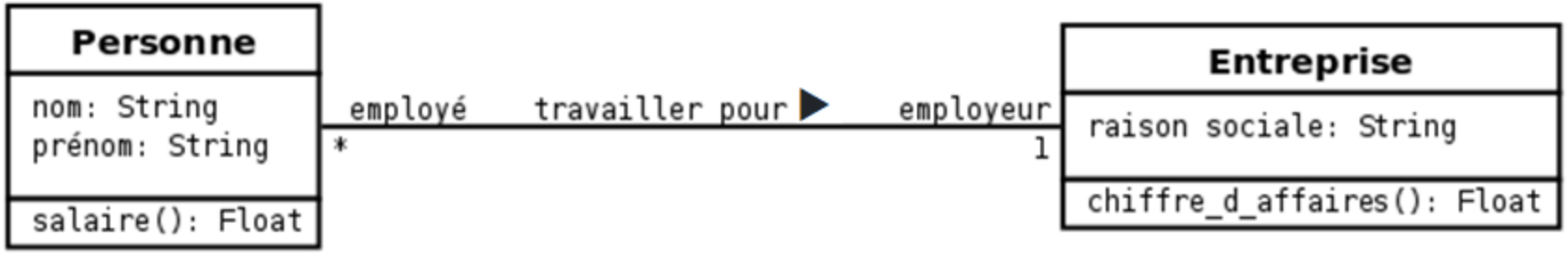
Relations entre classes

Notion d'association

- **Association** = **relation** entre **deux classes** (association binaire) **ou plus** (association n-aire), qui décrit les **liens** structurels entre les **instances des classes** (objets).

- Une **association** est représentée par un **trait** entre classes.

- ***Exemple*** : Association entre les classes « *Personne* » et « *Entreprise* » :



- Une **association** est caractérisée par son : ***Nom***, ***Sens de lecture***, ***Cardinalités***, ***Rôles***, ***Visibilité des rôles***, ***Navigabilité***.

Relations entre classes

Nommage des associations et des rôles

- Une **association** est **nommée** avec éventuellement une précision du **sens de lecture** (▶ ou ◀).
- Le **nom de l'association** est noté par une **forme verbale** (active ou passive) et il est écrit en italique au ***milieu*** de la ligne.
- De même, les **rôles** de l'**association** peuvent être **nommés** afin de définir la manière dont une classe intervient dans une **relation**.

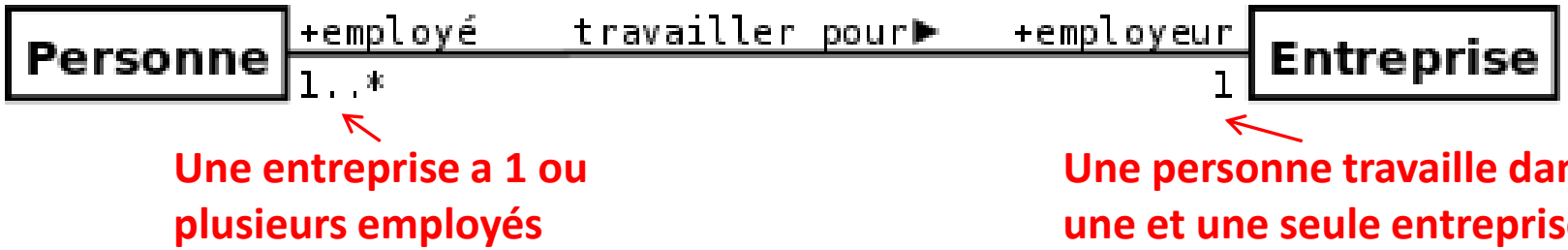


Relations entre classes

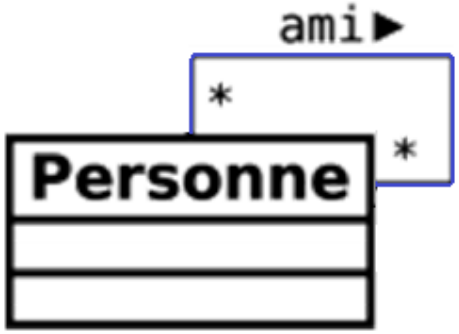
Association binaire

- Une **association binaire** lie **deux classes** et elle est matérialisée par un **trait plein** entre les **classes** associées.

- ***Exemple*** d'une association binaire :



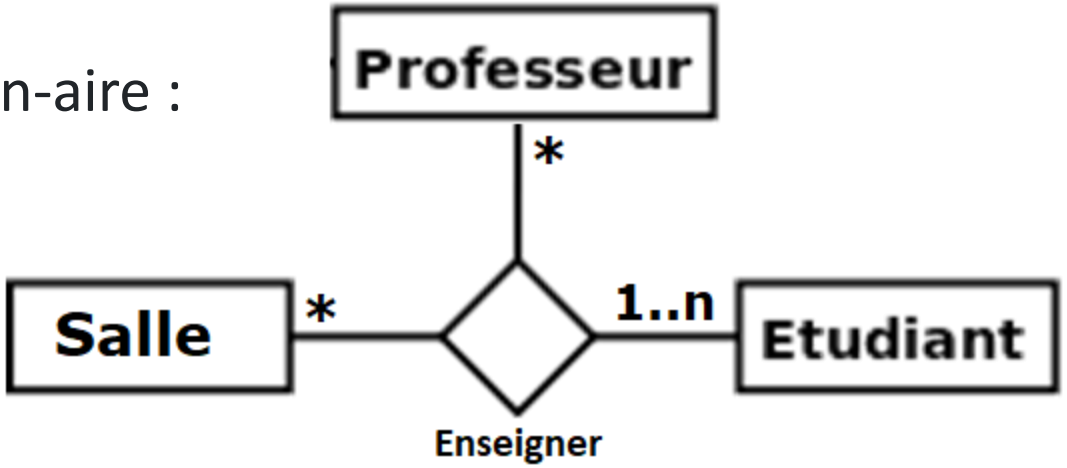
- Quand les deux extrémités de l'association pointent vers la **même classe**, l'**association** est dite ***réflexive***.



Relations entre classes

Association n-aire

- Une **association n-aire** lie **plus de deux** classes. Elle est représentée par un **losange** avec un trait partant vers chaque classe participante.
- Le **nom** de l'association apparaît à proximité du losange.
- ***Exemple*** d'une association n-aire :

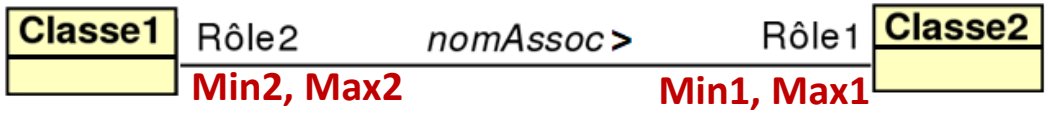


- ***Remarques*** :
  - Les **associations n-aires** sont peu fréquentes. On utilise des **classes-association** ou plusieurs **associations binaires**.

Relations entre classes

Cardinalité des associations

- La **cardinalité** d'une association précise combien d'**instances** d'une **classe** peuvent se rattacher a une seule **instance** d'une autre classe.



- Les cardinalités sont représentées par la notation : **Min …. Max**.
- Les cardinalités proposées par UML sont :

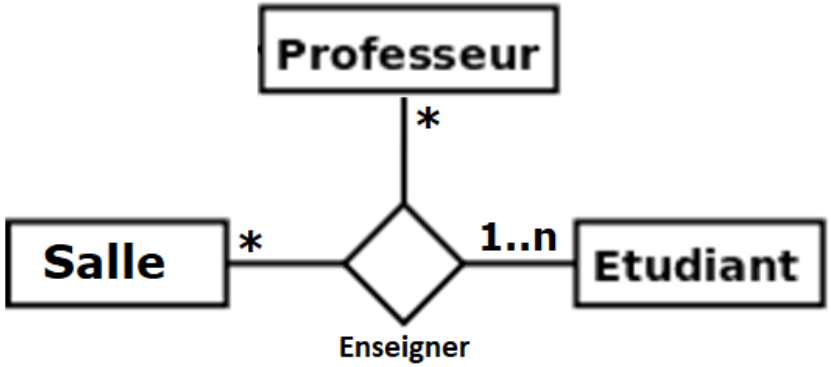
Spécifications	Cardinalité
1 ou 1..1	Exactement 1
0..1	Au plus un
* ou 0..*	0 ou plusieurs
1..*	1 ou plusieurs
n..m	Entre n et m
n..m, p	n, m ou p

Relations entre classes

Cardinalité des associations

- Dans une **association n-aire**, la **cardinalité** apparaissant sur le lien de chaque **classe** s'applique sur **une instance** de chacune des autres classes participantes.

- ***Exemple*** : Soit l'**association ternaire** entre les classes **Etudiant**, **Salle** et **Professeur**.

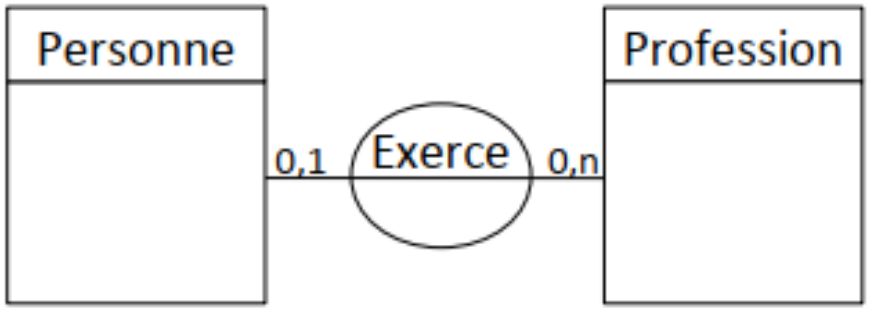


➔ La cardinalité de la terminaison **Professeur** indique le nombre d'objets **Professeur** qui peuvent apparaître dans l'association, avec **un pair** particulier d'objets (**Etudiant**, **Salle)**.

Relations entre classes
Cardinalité des associations

- ***Remarque*** :
  - En **UML**, et par référence à la méthode **Merise**, les **cardinalités** sont **«à l'envers»** pour les **associations binaires** et **«à l'endroit»** pour les **associations n-aires** (n>2).
  - ***Exemple*** : Cardinalités en **Merise** et en **UML** :

Merise



MCD

UML

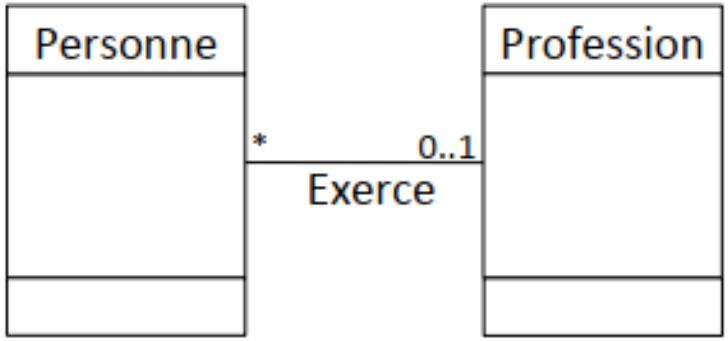
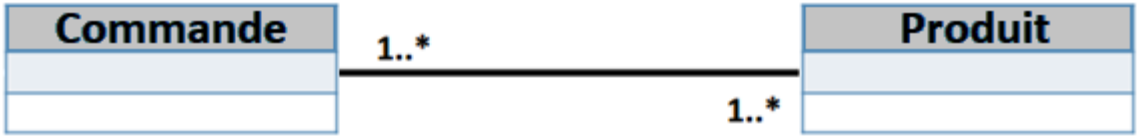


Diagramme de classes

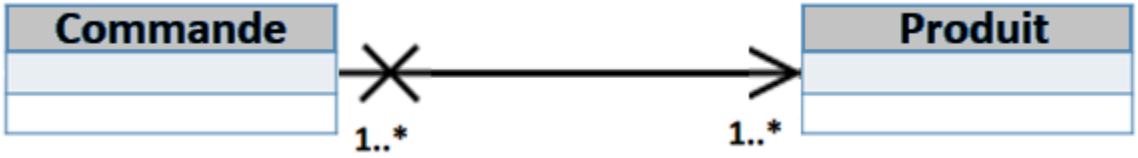
Relations entre classes

Navigabilité d'une association

- Par défaut, une association peut être traversée dans les deux sens.



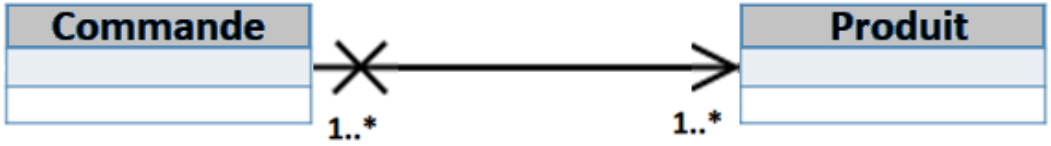
- Il est possible de **se limiter** à une seule direction➔ **navigabilité**.
- La **navigabilité** indique s'il est possible de **traverser** une **association**.
- Graphiquement, la **navigabilité** est représentée par une **flèche** du côté de la **terminaison navigable**.
- On empêche la **navigabilité** par une **croix** du côté de la **terminaison non navigable**.



Relations entre classes

Navigabilité d'une association

- Réduire la portée de l'**association** indique que les **instances** d'une classe "ne connaissent pas" les **instances** de l'autre classe.
- ***Exemple*** : Soit le diagramme de classes suivant :

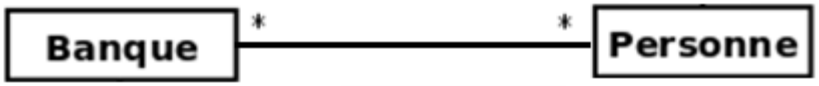


➔Connaissant une **Commande** on connaît les **Produits**, mais pas l'inverse.

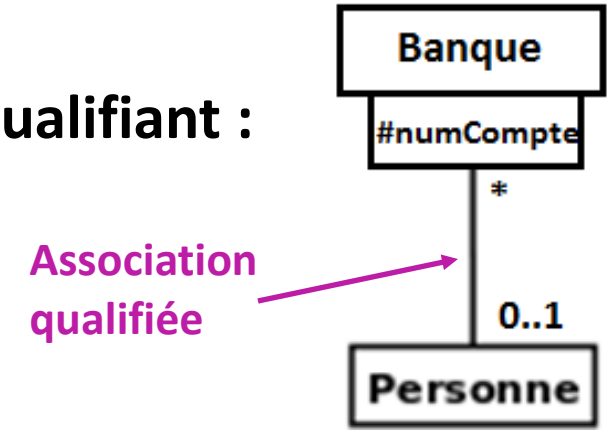
- ***Remarques*** :
  - Pour une **association à navigabilité restreinte**, les 3 associations suivantes sont équivalentes : 
  - On peut aussi représenter les **associations navigables dans un seul sens** par des **attributs** (dans l'exemple ci-dessus, on ajoute dans la classe **Commande** un attribut de type ***Produit***).

Relations entre classes
Qualification d'une association

- La **cardinalité** d'une association précise combien d'**instances** d'une **classe** peuvent se rattacher a une seule **instance** d'une autre classe.



- La **qualification** d'une **association** permet de **restreindre** la cardinalité de l'association : on ne sélectionne qu'un **sous-ensemble d'objets** parmi les objets qui participent à l'association.
- La restriction se fait sur la base d'un ou plusieurs attributs, dits **qualifiants**.
- ***Exemple*** : le ***numCompte*** est un **attribut qualifiant :**

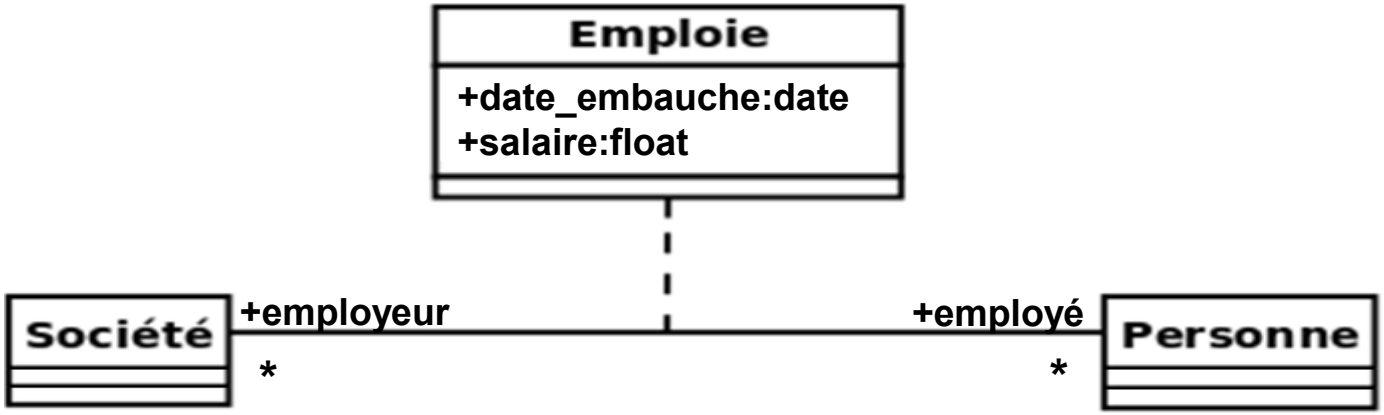


➔ une instance du couple {***Banque, numCompte***} est en association avec une et une seule instance de la classe ***Personne***.

Relations entre classes

Classe association

- Parfois, l'**association** doit posséder des **propriétés**. Pour modéliser cette situation, on utilise le concept de **classe-association**.
- Une c**lasse-association** est représentée par un trait discontinu entre la **classe** et l'**association** qu'elle représente.
- ***Exemple***: l'association "***Emploie***" entre les classes «***Société***» et «***Personne***» possède les attributs "***salaire***" et "***date d'embauche***".

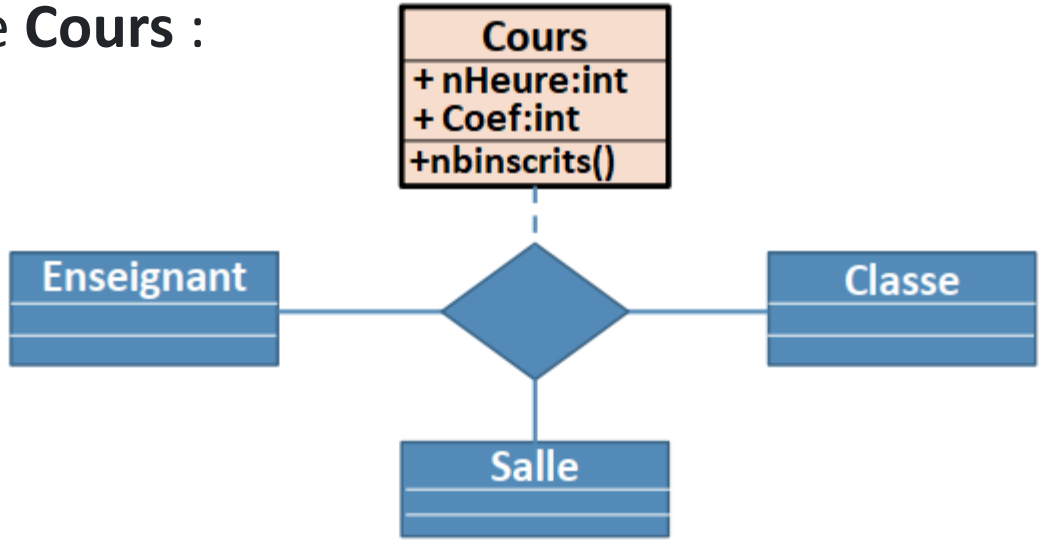


- L'association "**Emploie**" est représentée à l'aide d'une **classe-association**.

Relations entre classes

Classe association

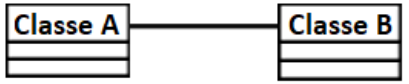
- Une **classe-association** possède à la fois les caractéristiques des :
  - **associations** : se connecte à deux ou plusieurs **classes**
  - **classes** : possède des **attributs** et des **opérations**.
- ***Exemple*** : Une **association ternaire** entre **Salle**, **Classe** et **Enseignant** est réifiée comme une classe **Cours** :



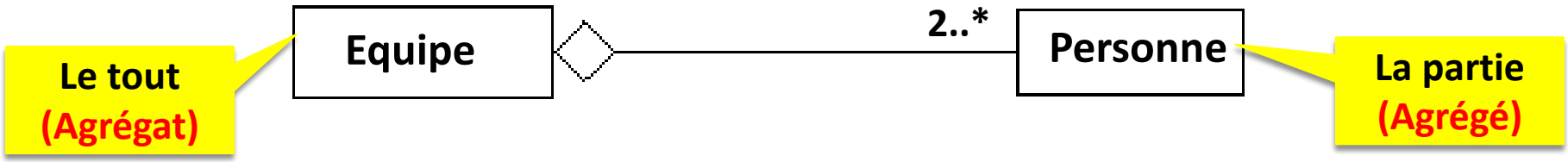
- ***NB*** : Il n'est pas possible de rattacher une **classe-association** à plus d'une **association**.

Relations entre classes

Agrégation

- Une **association** simple entre deux **classes** représente une **relation symétrique** (aucune des deux classes n'inclut l'autre). 

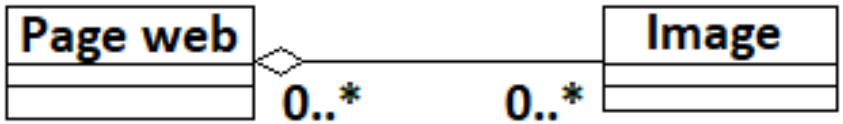
- Une relation de type **ensemble/élément** où une **classe** constitue un **élément** d'une autre **classe**, est modélisée par une **agrégation**.
- L'**agrégation** est une relation d'**inclusion** d'un ***élément*** dans un ***ensemble*** (tout/partie) .
- Graphiquement, on ajoute un **losange** vide (◇) du côté de l'**agrégat**.
- ***Exemple*** : relation d'agrégation :



Relations entre classes

Agrégation : Quand est-ce-que on l'utilise ?

- On utilise l'**agrégation** :
  - s'il y a une **asymétrie intrinsèque** dans la **relation** : l'**image** fait partie de la **page Web**, mais la **page Web** n'est pas une partie de l'**image**.



  - si la destruction de l'**agrégat** n'entraîne pas celle de l'**agrégé**, mais seulement la suppression du **lien**.

  - si des **opérations** sur « **le tout** » sont automatiquement appliquées à « **la partie** » (fermer la page entraine la fermeture de l'image).

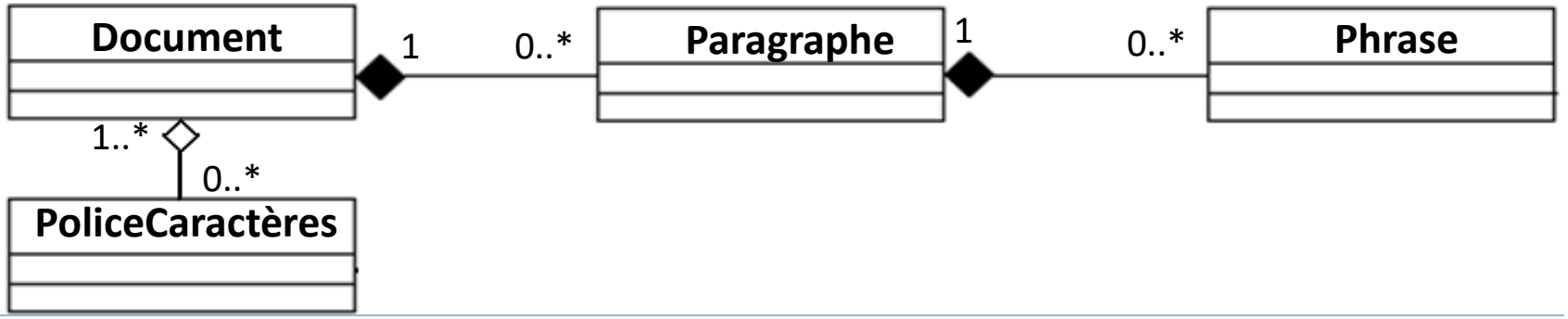
Relations entre classes

Composition (agrégation composite)

- La **composition** est un cas particulier de l'**agrégation** dans laquelle la destruction de l'**objet composite** implique la destruction de **ses composants.** Elle se représente par un losange plein (◆) .

- Contrairement à l'**agrégation**, une instance du **composant** ne peut être liée qu'a un seul **agrégat**: la **multiplicité** du côté composite est **1** ou **0..1**.

- ***Exemple*** : Un document est **composé** de plusieurs paragraphes, qui, à son tour, est composé de plusieurs phrases.

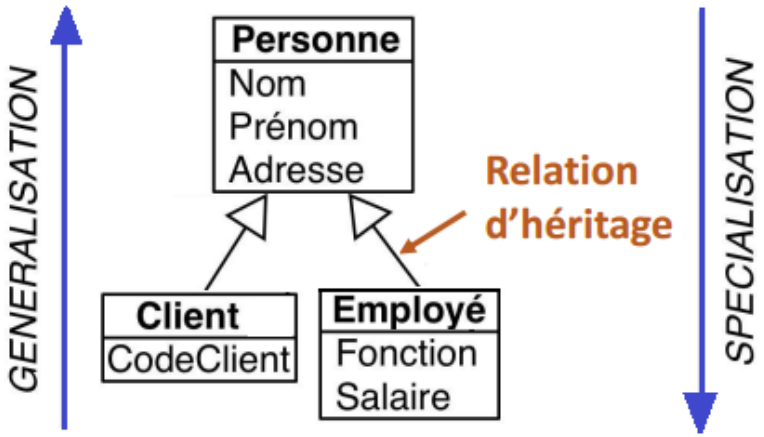


Relations entre classes

Héritage (Généralisation/Spécialisation)

- L'**héritage** est une relation de **spécialisation/généralisation** entre 2 classes. Il est représentée par une flèche suivante ⟶
- ⇨ la **classe mère** **généralise** la **classe fille**.
- ⇨ la **classe fille** **spécialise** la **classe mère**.

- ***Exemple*** :



- **Généralisation** : « **Client** » et « **Employé** » sont des classes qui ont des **attributs** en **commun**, on les factorise dans une classe « **Personne** ».
- **Spécialisation** : à partir de la classe « **Personne** » on se spécialise pour décrire de nouveaux détails aux classes « ***Client*** » et « **Employé** ».

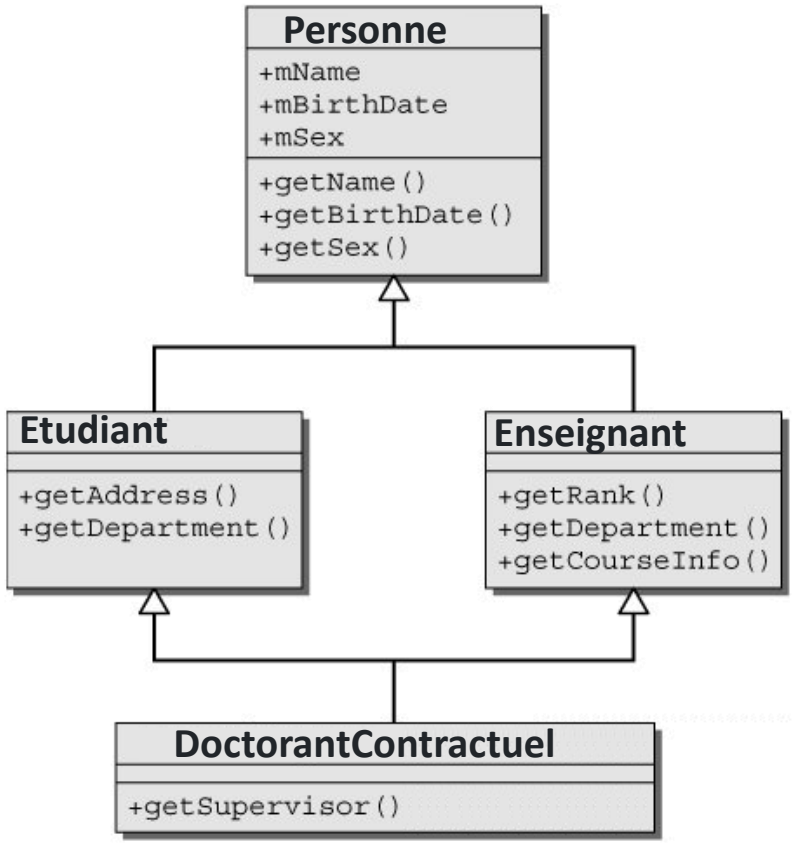
Relations entre classes

Héritage : héritage multiple

- Une classe peut avoir **plusieurs parents**, on parle alors d'**héritage multiple** (permis en C++, mais Java ne le permet pas).

- ***Exemple*** :

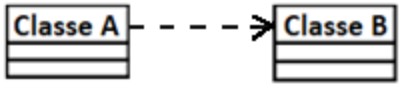
  Un **DoctorantContractuel** est un étudiant qui assure des cours.



Relations entre classes

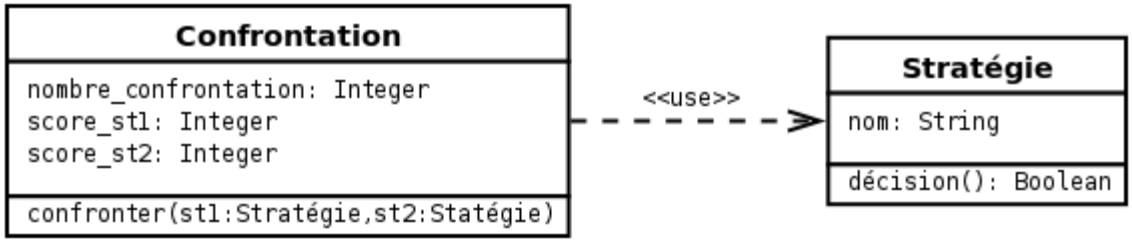
Dépendance

- Une **dépendance** est une **relation unidirectionnelle** exprimant une **dépendance sémantique** entre des **éléments** du modèle.
- La **dépendance** indique que la modification de la **cible** peut impliquer une modification de la **source**.



- Elle est représentée par une **flèche ouverte pointillée** avec un **stéréotype** explicitant la sémantique du lien (use, instanceof, …).
- On utilise souvent une **dépendance** quand une classe en utilise une autre comme argument dans la signature d'une opération.
- ***Exemple*** : la classe "*Confrontation*" utilise la classe "*Stratégie*" :

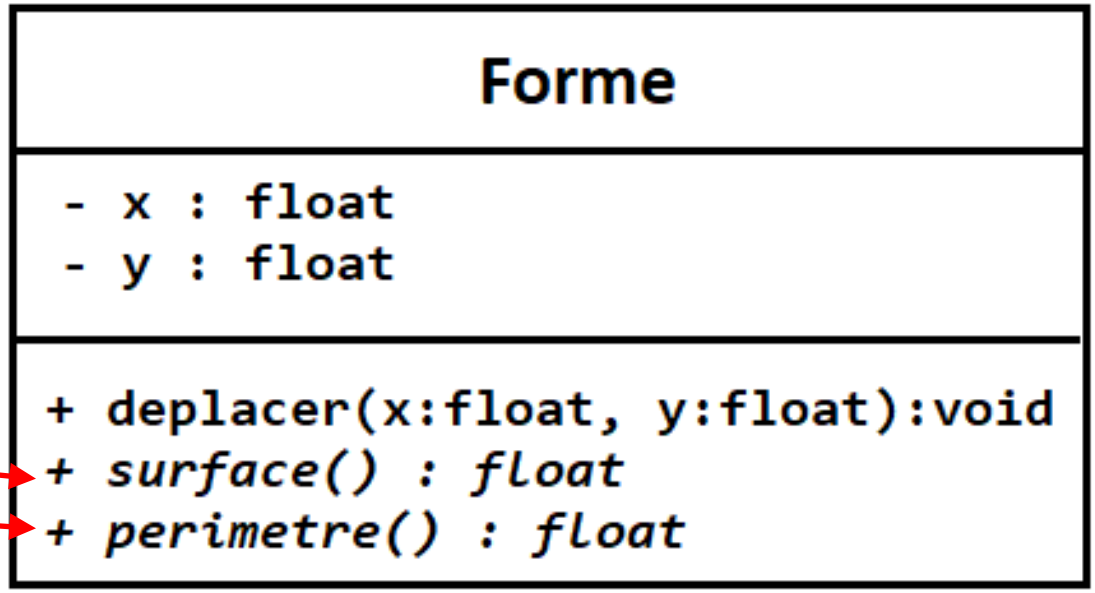
Si la *Stratégie* change, idem pour la *Confrontation*.



Relations entre classes

Méthode abstraite

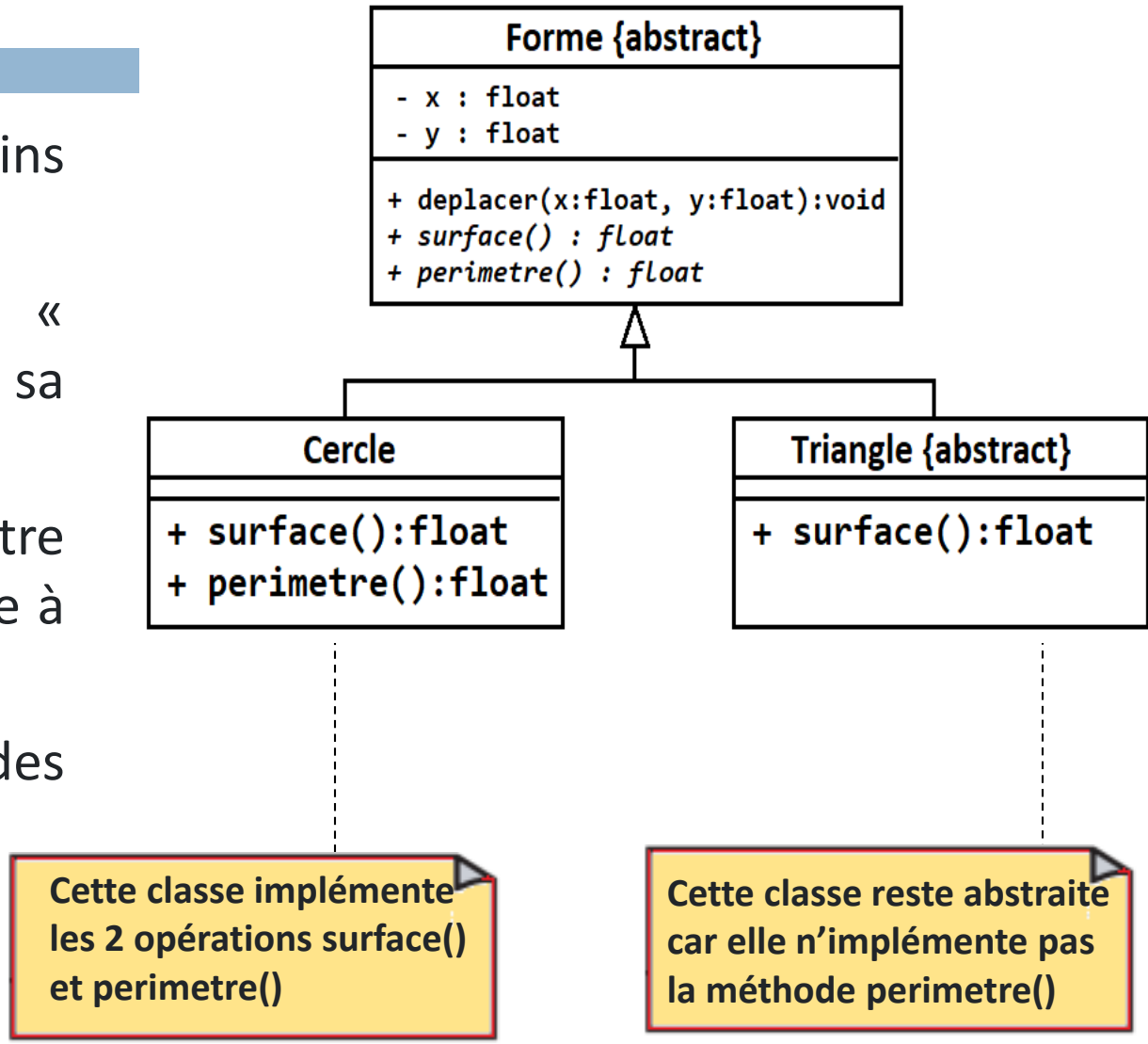
- **Méthode abstraite : méthode** dont on connaît sa **signature**, mais pas **sa définition**.
  - Ce sont les **classes filles** qui définissent les méthodes abstraites.
  - En UML, on écrit son nom en ***italique***.
- ***Exemples*** :



Relations entre classes

Classes abstraites

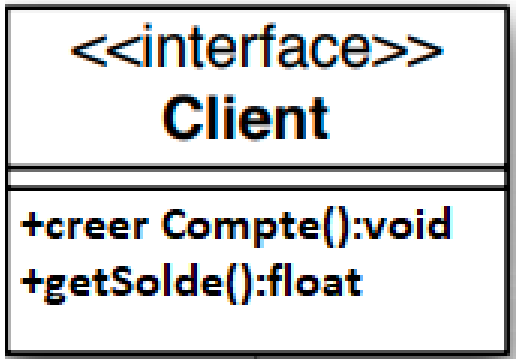
- **classe** qui définit au moins une **méthode abstraite**.
- On utilise le mot clé « ***abstract*** » dans sa définition.
- Elle ne peut pas être instanciée. Elle est vouée à se spécialiser.
- Elle peut contenir des **méthodes concrètes**.



Relations entre classes

Interfaces

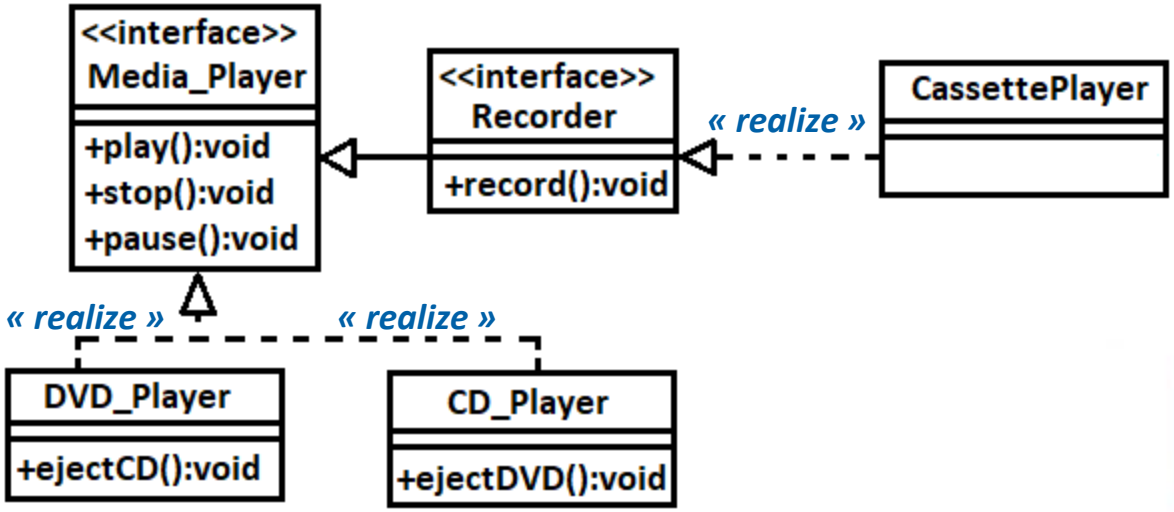
- Une **interface** est classe qui :
  - ne comporte que des **méthodes abstraites**.
  - Ne possède pas d'attributs mais peut déclarer des **constantes**
  - ne peut pas être **instanciée**.
  - Représentée en ajoutant le stéréotype **<<interface>>.**
- **Intérêt :** regrouper plusieurs classes par polymorphisme.
- ***Exemple*** : interface représentant les clients d'une banque :



Relations entre classes

Interfaces

- Une **interface** doit être **réalisée** par au moins une **classe**.
- la **classe** implémentant une **interface** doit implémenter **toutes les opérations** de l'**interface** (sinon elle reste **abstraite**).
- La **relation** de **réalisation** est représentée par une flèche d'héritage en pointillé avec le stéréotype « ***realize*** ».

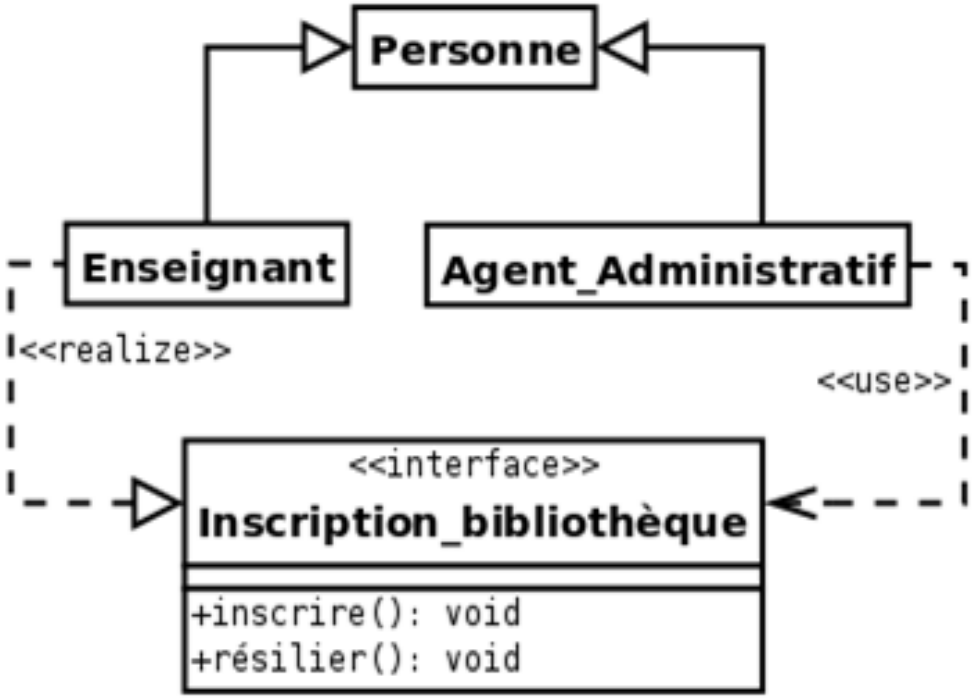


- Une classe peut réaliser plusieurs interfaces
- Une interface peut hériter d'une autre interface

Relations entre classes

Interfaces : Classe cliente d'une interface

- Une **classe** peut **dépendre** d'une **interface** (interface requise) pour **réaliser** ses opérations (elle est dite **classe cliente de l'interface)**.
- Dans ce cas, on utilise une relation de **dépendance** entre la **classe cliente** et l'**interface** (représentée cela avec le stéréotype « *use* »).
- ***Exemple*** :



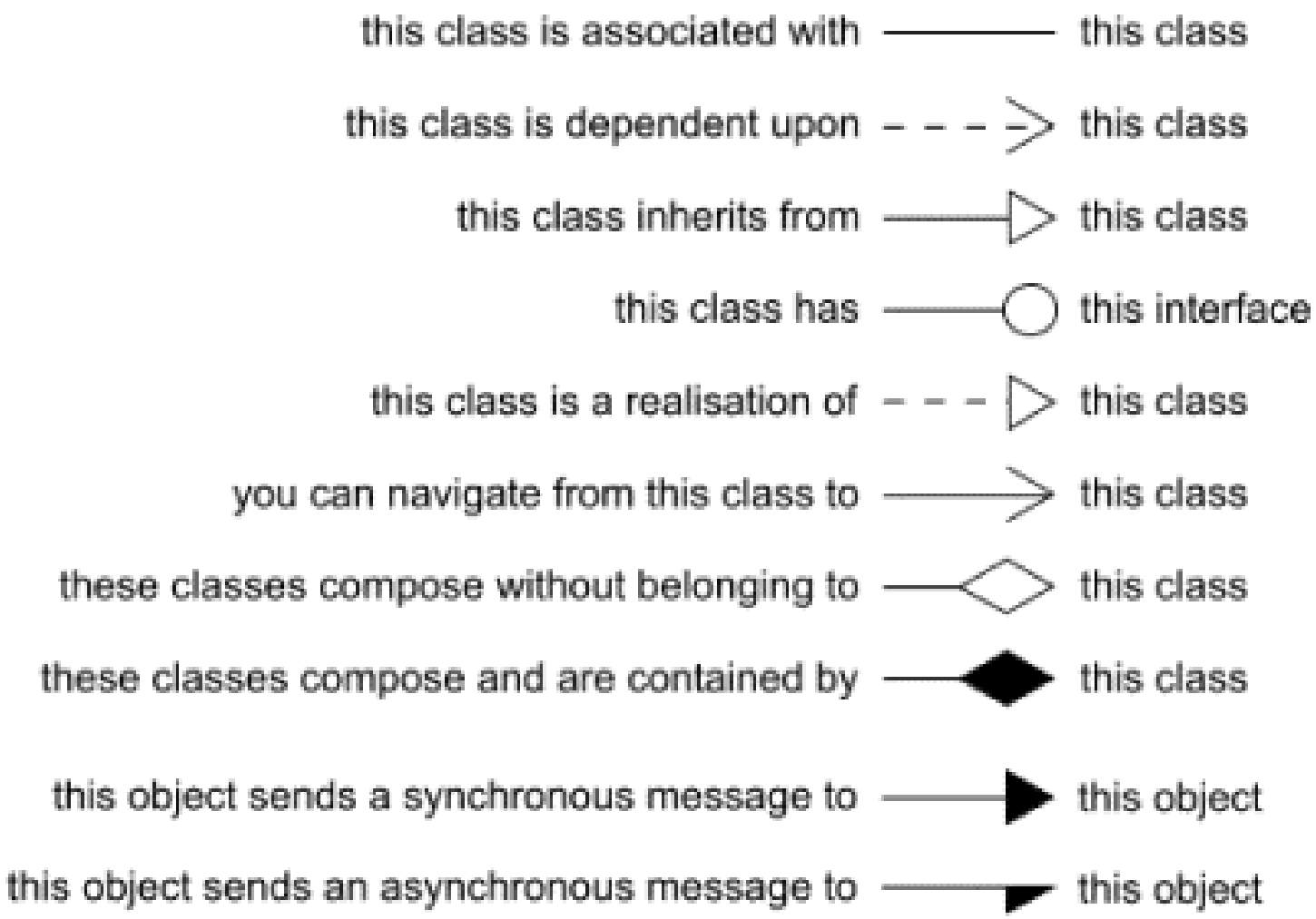

this class is associated with
this class
this class is dependent upon
this class
this class inherits from
this class
this class has
this interface
this class is a realisation of
this class
you can navigate from this class to
this class
these classes compose without belonging to
this class
these classes compose and are contained by
this class
this object sends a synchronous message to
this object
this object sends an asynchronous message to
this object

Diagramme de classes

Exemple

- Diagramme de classes d'un DAB (Distributeur Automatique Banquaire) :

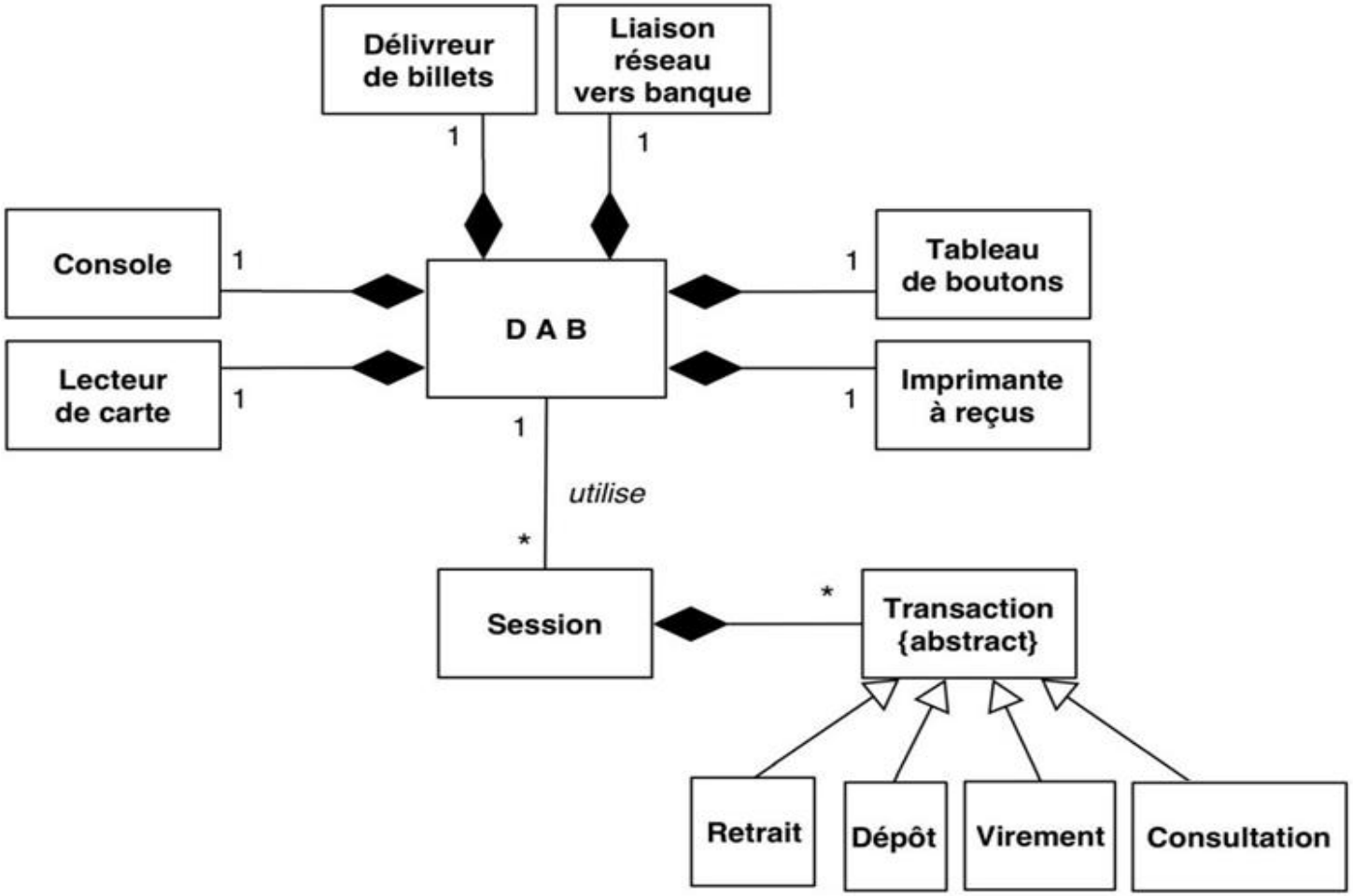


Diagramme d'objets

(Objects Diagram)

Diagramme d'objets

Introduction

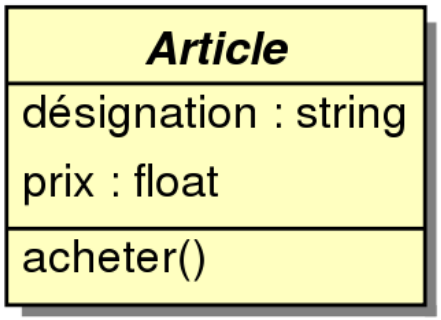
- Le **diagramme d'objets** représente des **objets** (instances de classes) et leurs **liens** (instances de relations) à un **instant donné**.
- Il peut être utilisé pour :
  - Illustrer le **modèle de classes** (en montrant un **exemple** qui explique le modèle).
  - Préciser certains **aspects** du système (en mettant en évidence des **détails** imperceptibles dans le diagramme de classes).
  - Exprimer une exception (en modélisant des **cas particuliers** qui ne sont pas modélisés dans un diagramme de classe).

Le **diagramme de classes** modélise des **règles**.
Le **diagramme d'objets** modélise des **faits**.

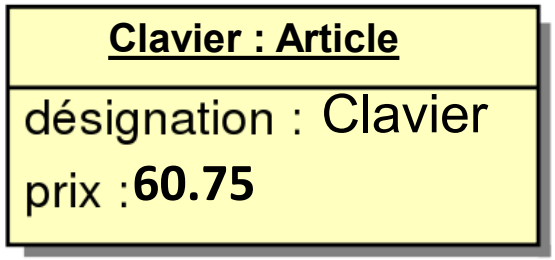
Diagramme d'objets

Représentation graphique

- Un **objet** est représenté comme une **classe**, mais le compartiment des **méthodes** n'est pas indiqué.
- Le **nom** de l'**objet** est composé du nom de l'**instance**, suivi de celui de la **classe**, et est **souligné**.



Classe



Instance (objet)

- Les **attributs** reçoivent des **valeurs** : si certaines valeurs ne sont pas renseignées, l'objet est **partiellement défini**.

Diagramme d'objets
Représentation graphique

- Un **lien** est une **instance** d'une **association**. Il est représenté comme une **association** mais s'il a un nom, il est souligné.
- La **relation** de **généralisation** n'est **jamais représentée**.
- Les **multiplicités** (cardinalités) ne sont pas représentées.
- Les relations de **dépendance d'instanciation** relient les ***associations*** aux ***liens*** et les ***classes*** aux ***objets***.
- Cette relation est stéréotypée: <<instanceof>>

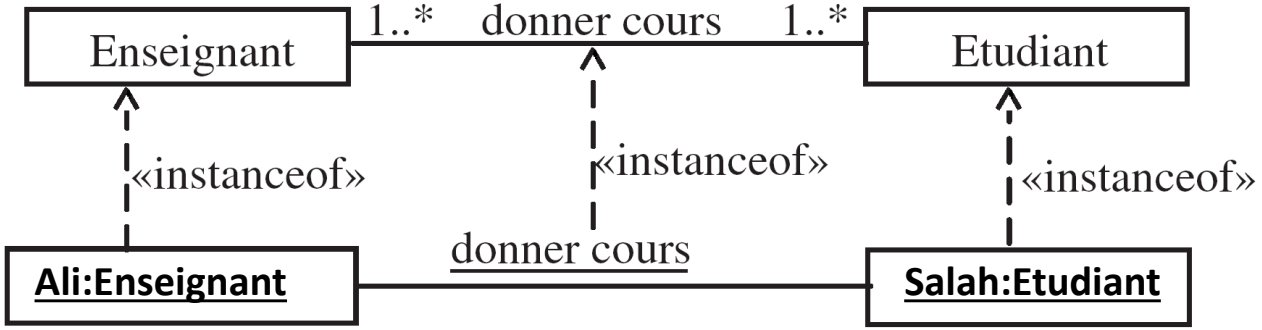
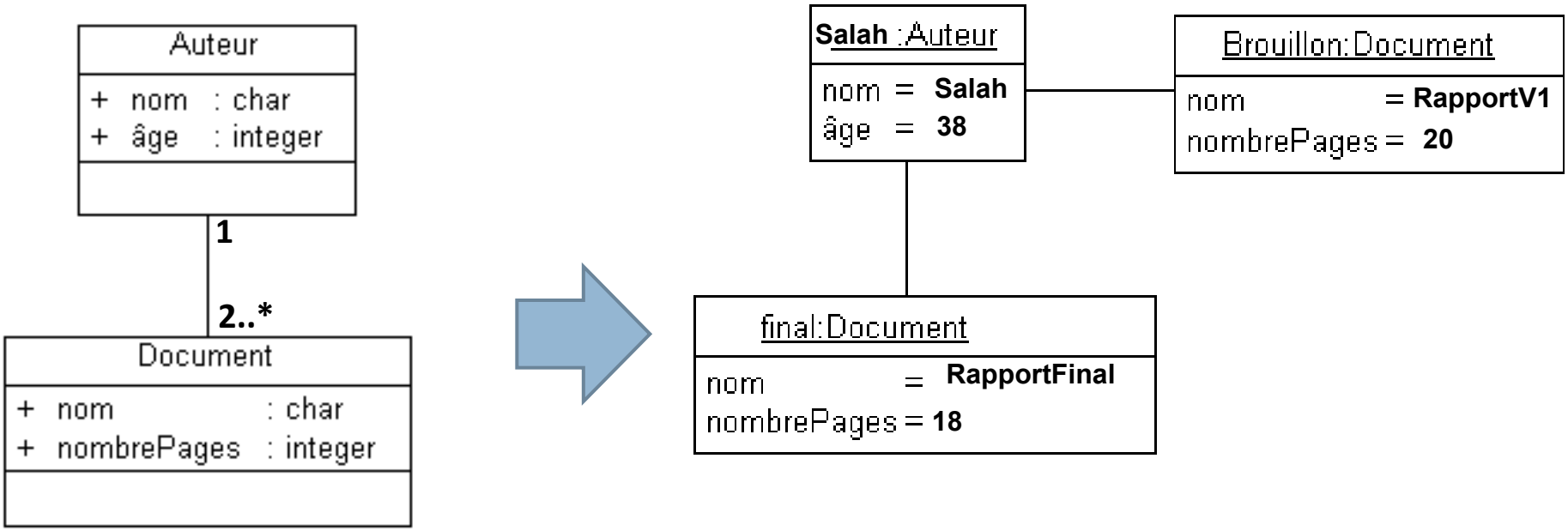


Diagramme d'objets

Exemple 1

- **Diagramme d'objets cohérent** avec le **diagramme de classes** :



- L'objet « **Salah** », instance de la classe « **Auteur** » est lié à deux objets différents (« **Brouillon** » et « **Final** ») qui sont deux instances de la classe « **Document** ».

Diagramme d'objets
Exemple 2

- **Diagramme d'objets incohérent** avec le **diagramme de classes** :

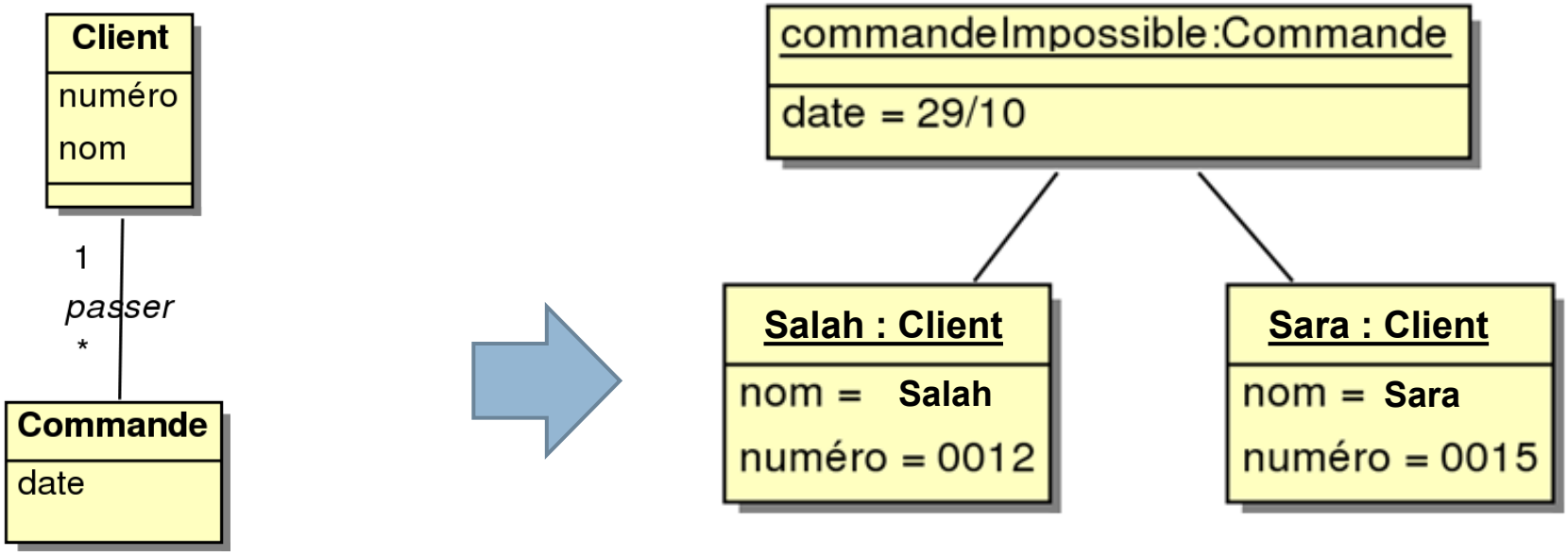


Diagramme de classes

Diagramme d'objets

- Une « **commande** » ne peut être passé que par **un et un seul client** !!

Élaboration d'un diagramme de classes

Élaboration d'un diagramme de classes
Démarche

- Une démarche couramment utilisée pour bâtir un diagramme de classes est la suivante :
  - **1**. Pour identifier les classes, souligner tous les *termes* décrivant le domaine applicatif (cours, salles, professeurs… pour une application gérant un Emploi du Temps) ;
  - **2**. Réduire l'ensemble obtenu avec les critères suivants :
    - supprimer des *synonymes*,
    - supprimer des classes *trop vagues* (elles sont surement ailleurs),
    - supprimer des classes non *pertinentes* pour l'application,
    - découvrir les *associations* exprimant l'interdépendance des classes, et les nommer ;
    - trouver les *attributs* des classes, vérifier que chaque attribut caractérise bien la classe dont on parle.
    - identifier les *opérations* des classes : association ou attribut ?

Élaboration et implémentation d'un diagramme de classes

- **3**. Inclure les relations de *généralisation/spécialisation*
- **4**. Placer des *agrégations/compositions* si nécessaire
- **5**. Factoriser avec des *interfaces*
- **6**. Utiliser les *contraintes*
- **7**. Réorganiser le diagramme de classes en construisant des *packages*
- **8**. Vérifier *l'atteignabilité* de tous les éléments (se poser des questions sur les fonctionnalités utilisateurs : par ex. pour une application marchande, est-ce que je peux connaître les éléments de mon panier, en transitant par les différentes associations de mon diagramme de classes ?)
- **9**. Valider le modèle obtenu avec les *utilisateurs*
- **10**. Incrémenter le modèle (en itérant) : on n'obtient jamais le bon diagramme de classes dès le 1er coup ; un indicateur de bon diagramme peut être le suivant : compter les versions obtenues. A partir de la 3ème, on peut penser être proche d'un diagramme correct.